Rerank, secure, measure

The last 3 pieces of production RAG · written for a cloud / infra person

1 Reranking in one picture

Retrieval gave us **5 chunks**, but we only want the **best 3** for the LLM. Their order isn't trustworthy (similar ≠ relevant).

Q + C1…C5 → Reranker (1 API call) → C1+Q 0.9 · C2+Q 0.1 · C3+Q 0.7 …

sort by score → keep top 3 → LLM

🔧 Cheap wide health check first (retrieval), then an expensive deep probe on the shortlist only (reranker).

Bi-encoder

Embeds query and chunk **separately**, compares vectors. Precomputed, fast over **millions**. Good at "same topic".

🔧 Matching CVs by keywords.

Cross-encoder

Reads query + chunk **together** and outputs one score. Slow, per pair, so use it on the **top 20–50** only. Great at "does it answer?"

🔧 The actual interview.

2 Security in one rule

Authorisation before evidence reaches the model. Public consumer docs and internal partner docs can look alike. If an internal chunk lands in the prompt, assume it **leaks**. The LLM is not a security boundary, so filter **before** search with metadata built on the server from the login.

🔧 RBAC + namespaces enforced by the API server. Deny by default.

3 Evaluation in one rule

Every enhancement (hybrid, rerank, rewrite…) must **beat the baseline on a golden set**. Measure **retrieval** and **generation** separately, so you know what to fix. 🔧 SLOs + a CI regression gate for answer quality.

Reranking & security words

Each term has a one-line meaning plus 🔧 an infra analogy

① Reranking

Reranker: a model that re-scores retrieved chunks against the question, then we keep the top K.

Relevance score: 0–1 per (chunk + query) pair. 0.9 = really answers it, 0.1 = off-topic.

Cross-encoder: reads both texts at once. Accurate, slow, no precompute.

Bi-encoder: the embedding model. Separate vectors, fast, precomputed.

Retrieve wide, rerank narrow: fetch 20–50, rerank, keep 3–5.
🔧 Wide cheap scan, then a deep check on the shortlist.

Voyage AI: the class's reranker provider (now part of MongoDB). Others: Cohere, BGE, Jina.

Rerank fallback: API timeout → use the RRF order. Low best score → abstain.
🔧 A circuit breaker with a degraded mode.

② Security

Trust boundary: a point where untrusted input enters (user, query rewrite, documents, model output).
🔧 A network perimeter, where you inspect what crosses.

Access metadata: tenant_id, audience (consumer / partner / internal), role, status on **every chunk**.

Server-built filter: `must` conditions from the **session**, never from the question.
🔧 IAM policy attached by the platform.

Deterministic security: plain-code filters, not LLM judgement. Same user → same rights.

Prompt injection: hidden instructions inside documents ("ignore previous rules…"). Treat retrieved text as **data**.
🔧 SQL injection, but for prompts.

Allowlisted filters: self-query may suggest topic/locale; tenant/role are never model-controlled.

Masked logs: prompts and answers in logs carry the same sensitivity as the data.

Evaluation words



Find out whether retrieval or generation broke

③ Retrieval metrics

Golden set: questions + the right chunk IDs + reference answers. Include traps: codes, follow-ups, other tenants, unanswerable.
🔧 A test fixture / smoke-test suite.

Recall@K: relevant found ÷ **all relevant**. "Did we find them?"

Precision@K: relevant found ÷ **K**. "How much noise?"

Hit rate@K: at least one right chunk in the top K? yes/no.

MRR: 1 ÷ rank of the first right chunk (rank 1 → 1.0, rank 3 → 0.33).

nDCG: rewards putting the best chunks **first**, which is exactly what reranking improves.

④ Generation metrics

Faithfulness: is every claim backed by the context? The hallucination metric.

Answer relevance: does it actually answer the question?

Context precision / recall: is the context mostly useful / complete?

Citation accuracy: cited IDs exist and support the claim.

Correct abstention: says "I don't know" for "laptop insurance?"

LLM-as-judge / RAGAS: an LLM scores answers against rubrics. Cheap at scale; spot-check it.

⑤ Ops metrics

p95 latency per stage: retrieval, rerank, LLM.
🔧 Per-hop tracing.

Cost per query: tokens + rerank calls + embeddings.

Fallback / error rate: how often you degrade or fail closed.

The whole RAG story

Chapters 01 → 04 · say this out loud once

"Walk me through production RAG"

"**Indexing:** parse (OCR for scans) → structure-aware chunks with tenant/audience metadata → optional contextualisation → dense + BM25 vectors in Qdrant. **Query:** router → trusted identity → faithful rewrite → **server-built access filter** → dense + BM25 → RRF → **cross-encoder rerank** → top 3–5 → grounded LLM at temperature 0 with citations, or abstain. **Around it:** timeouts and fallbacks that fail closed, plus a **golden-set evaluation** in CI for every change."

Q Quick answers

Why rerank?

Retrieval finds candidates; the reranker orders them so the right one survives top-K.

Bi vs cross encoder?

Separate vectors (fast) vs reading both together (accurate). Use both, in that order.

Rerank API down?

Timeout → RRF order → log it.

Secure RAG?

Filter before search from the session's identity. The LLM is not a boundary.

Prompt injection?

Docs are data, not instructions; delimit them; vet sources.

Evaluate RAG?

Retrieval (Recall@K, MRR, nDCG) + generation (faithfulness, relevance) + ops.

Faithfulness?

Every claim supported by the context.

Ship a change?

Only if it beats the baseline on the golden set.

The 4 chapters

01 Fundamentals	embeddings · similarity · Qdrant · HNSW
02 LangChain	loaders · splitters · retriever · LCEL
03 Production	7 failures · chunking · hybrid + RRF · query intelligence
04 Finish line	rerank · security · evaluation

RAG track ✓